

Prompt Log — Manual k-NN / Weighted k-NN Project

This document records the user prompts from this conversation in chronological order. It focuses on the development of the modular k-NN implementation, preprocessing, weighted k-NN, model interface, unit testing, and performance comparison.

Prompt 1 — Stratified Train/Test Split

I have an excel sheet with some data in it. The first task is to split the data into train and test dataset with a split of 70:30. I need to keep the ratio of the final target classes (CONDITION & NO CONDITION) the same after the split. Make sure that the same split is obtained for every run. The function should return `X_train`, `X_test`, `y_train` and `y_test`. Do not put print statements inside the function and keep the function modular. Perform this task under the function `ai_data_split(X,Y)`. Give me the python code for this task

Prompt 2 — Manual k-NN Implementation

Next lets create another python code for k-NN algorithm. First define a function called `ai_euclidean_distance()` to calculate the Euclidean distance between a training feature vector and a test feature vector. Then create another function named `ai_calculate_distances()`, which will use the `ai_euclidean_distance()` function to find out the distance between one test vector and all vectors in the training set and stores the distance and the index of the training vector. Then create a function called `ai_sort_distances()` which sorts these distance-index pairs in ascending order of distance and uses the appropriate method (you make the decision) as the tie breaker when two distances are equal. The sorting algorithm is Quick sort. After this create a function called `ai_knn_neighbors()` which takes the sorted distances and `k` and returns the `k` nearest neighbours. Next, define another function named `ai_majority_knn()` that accepts the chosen neighbours and `y_train` as inputs, then determines the class of each neighbor and returns the class that occurs the maximum number of times among the neighbours. In case of a tie, choose the appropriate method or technique to help solve the issue. Finally create a function called `ai_knn_predict()` which takes `X_train`, `y_train`, `X_test` and `k` and uses all the above functions to predict the class of every test vector and return the predictions (count of each target class in the test dataset). Dont use `sklearn.neighbors.KNeighborsClassifier` or any other ready-made k-NN classifier. Keep all the functionality modular and do not put print statements inside the functions.

Prompt 3 — Missing Data and Encoding

The dataset has to be preprocessed first before we can apply the knn algorithm. So give me the modular function codes for handling missing data (as `ai_missing_data()`) and then encoding of the nominal and ordinal features as `ai_label_encoding()` and `ai_onehot_encoding()`

Prompt 4 — Weighted k-NN

Help me create another Python code for weighted k-NN where we use the same Euclidean distance approach and define a function called `euclidean_distance()`, then a function called `ai_wknn_calculate_distances()` which calculates the distance from every training vector to a given test vector and stores the distance and the index of the training vector. After that create a function called `ai_wknn_sort_distances()` to sort the distances in ascending order and uses the appropriate method (you make the decision) as the tie breaker when two distances are equal. The sorting algorithm is Quick sort. Then create a function called `ai_wknn_neighbors()` which selects the `k` nearest neighbours. Next create a function called `ai_class_weights()` which takes the selected neighbours and `y_train` and calculates the weight of every neighbour using $\text{weight} = 1/\text{distance}$ and then adds the weights belonging to the same class so that the total weight for each class can be obtained. Then create a function called `ai_weighted_majority()` which selects the class having the highest total weight and choose the appropriate method or technique as the tie breaker if two classes have the same total weight. Finally create a function called `ai_wknn_predict()` which performs weighted k-NN classification for every test vector using all these functions and returns the predictions. Do not use a ready-made weighted k-NN package or `sklearn.neighbors.KNeighborsClassifier` and do not put print statements inside the functions.

Prompt 5 — Fit, Predict and Score

Help me write another Python code where I will define the three functions to get the fit, predict and score of the k-NN model. First create a function called `ai_fit()` which takes `X_train` and `y_train` and stores the training feature vectors and their corresponding class labels in the appropriate form to use them in the k-NN prediction function. Then create a function called `ai_predict()` which takes `X_test`, training data, `y_train` and `k` as parameters makes use of functions defined in `ai_knn.py` file to perform prediction for all test vectors. The prediction function should not include another k-NN algorithm implementation but reuse those functions which were implemented in `ai_knn.py` file. Then create a function called `ai_score()` to calculate the accuracy of predictions as a number of correctly predicted test observations divided by the number of all test observations. Each function should return its required result and there should be no print statements inside the functions.

Prompt 6 — Add Weighted k-NN Prediction

i want to add weight kNN prediction function too in this file

Prompt 7 — Unit Test Structure

ok now for the files created, which is `ai_knn` and `ai_wknn` files, lets create unit test cases for the important tasks done. Lets make a separate python file that imports all these python files and makes separate classes to test the modular functions. The unit test cases should include normal cases, boundary cases and appropriate edge cases. Maybe we can use python's built-in `unittest` module for this code. Lets come up with a structure first

Prompt 8 — Unit Test Code

The structure looks good....so can you give me the codes for this python file now

Prompt 9 — Quick Sort Testing Scope

for the sorting algorithms lets test only quick sort...others u can leave it

Prompt 10 — Performance Comparison

Now, help me create another python code for performance comparison of 3 knn implementation. They are my self written k-NN implementation, the package k-NN implementation using `sklearn` and the AI-generated k-NN implementation from `ai_knn.py`. First define a function called `accuracy()` which takes the actual test class labels and predicted class labels and calculates the accuracy as the number of correctly predicted observations divided by the total number of test observations. Then define another function called `precision()` which calculates the precision of the predictions, then a function called `recall()` which calculates the recall, and then a function called `f_score()` which calculates the F-score using the appropriate precision and recall values. a function called `recall()` which calculates the recall, and then a function called `f_score()` which calculates the F-score using the appropriate precision and recall values. Finally define a function called `execution_time()` which measures the computational time required to perform the prediction for a given k-NN implementation. Keep all five functions modular and do not put print statements inside these functions. In the main comparison part load and preprocess the thyroid dataset and import `train_test_split` for all three k-NN implementations. Use the existing `ai_data_split()` function for the 70:30 stratified split. Compare the Lab 5 self-written k-NN implementation, the Lab 5 package k-NN implementation and the AI-generated k-NN implementation from `ai_knn.py`. Use `k` values of 7, 9 and 11 and perform the comparison separately for each `k` value. For every `k` value, run each of the three k-NN implementations 10 times and for every run calculate accuracy, precision, recall, F-score and execution time. Store the result of every run and after all 10 runs calculate the average value of each metric for each implementation

Prompt 11 — Verify Test Case Categories

for the test cases python file does it include all the three types of cases: normal, boundary and edge case.

Prompt 12 — Prompt Log Request

can you create a prompt log for this chat. Give me a downloadable pdf